Module 3

UVM Basics

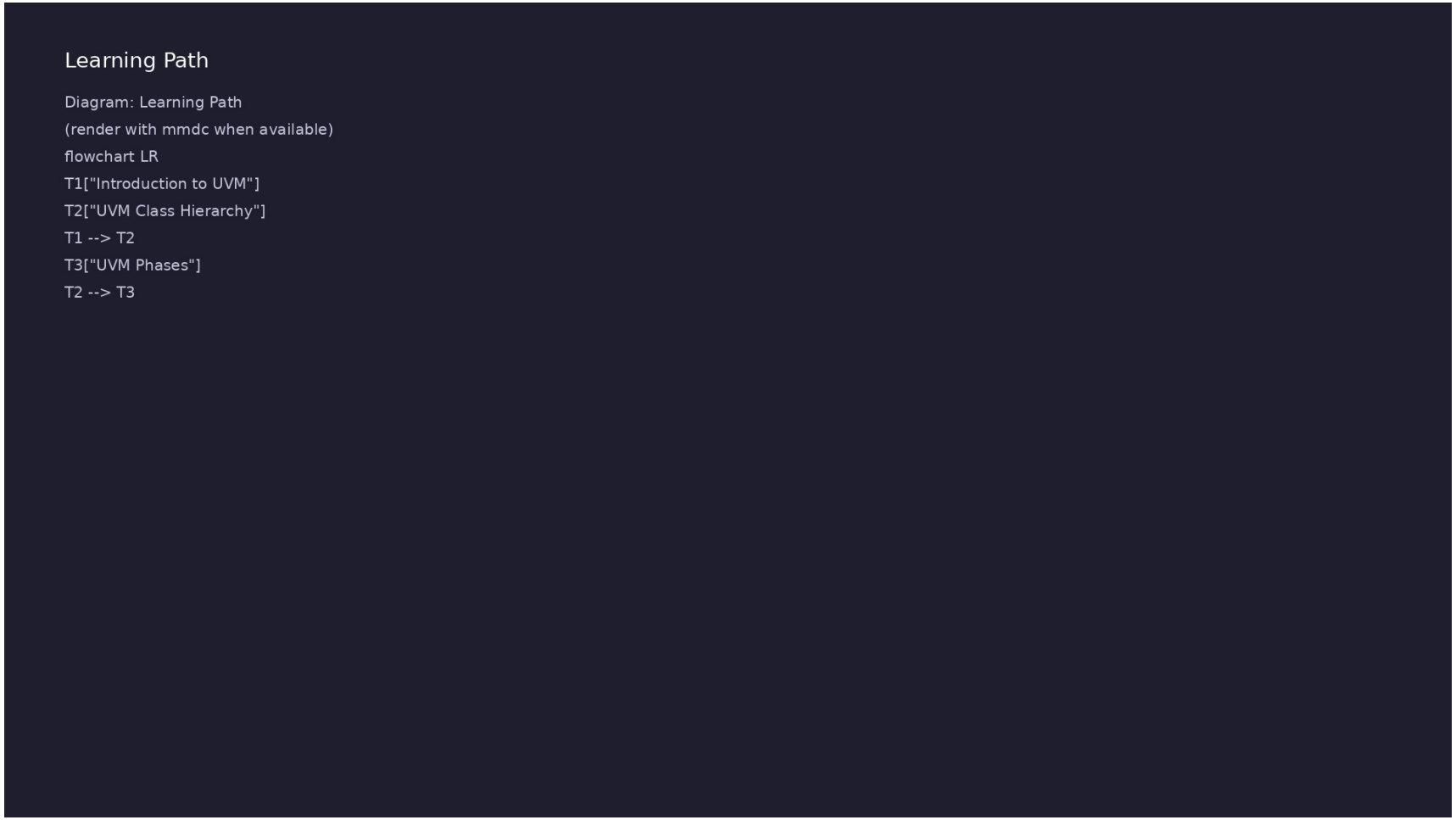
Learning objectives

- Master UVM class hierarchy and phases

Prerequisites

- See module README

Learning path



Master UVM class hierarchy and phases

Overview

- This module introduces the Universal Verification Methodology (UVM) using SystemVerilog. You'll lea...

Syllabus topics

1. Introduction to UVM (1/8)

- Universal Verification Methodology
- UVM standard (IEEE 1800.2-2017)
- UVM class library
- Methodology benefits
- Industry adoption
- UVM Benefits

1. Introduction to UVM (2/8)

- Reusability
- Standardization
- Scalability
- Maintainability
- Transaction Class (`MyTransaction`): Extends `uvm_sequence_item` (uvm_object hierarchy)
- Driver Class (`MyDriver`): Extends `uvm_driver` (uvm_component hierarchy)

1. Introduction to UVM (3/8)

- Monitor Class (`MyMonitor`): Extends `uvm_monitor` (uvm_component hierarchy)
- Agent Class (`MyAgent`): Extends `uvm_agent`, demonstrates component composition
- Environment Class (`MyEnv`): Extends `uvm_env`, top-level verification environment
- Test Class (`ClassHierarchyTest`): Extends `uvm_test`, orchestrates test execution
- Purpose: Automatically register class with UVM factory
- Usage: Must be inside class definition

1. Introduction to UVM (4/8)

- Key: Enables factory-based object creation
- Purpose: Initialize UVM objects and components
- Parameters:
- `name`: Component/object name
- `parent`: Parent component (for hierarchy)
- Key: Must call `super.new()` to initialize base class

1. Introduction to UVM (5/8)

- Purpose: Implement UVM phase behavior
- Key: Must call `super.build_phase(phase)` etc. to maintain hierarchy
- Objections: Use in `run_phase()` to control simulation
- Purpose: Create objects/components using factory
- Syntax: `ClassName::type_id::create(name, parent)`
- Key: Factory enables overrides and centralized creation

1. Introduction to UVM (6/8)

- Purpose: Connect component ports in `connect_phase()`
- Usage: Connect driver to sequencer, monitor to scoreboard, etc.
- Purpose: Provide human-readable string representation
- Usage: Debugging, logging, reporting
- `uvm_object` - Base class for all UVM objects (transactions, sequences)
- `uvm_component` - Base class for all UVM components (drivers, monitors, agents)

1. Introduction to UVM (7/8)

- Component hierarchy: Test → Environment → Agent → Driver/Monitor
- Component creation using factory pattern
- Factory macros (`uvm_object_utils`, `uvm_component_utils`) enable registration
- Phase functions/tasks implement component behavior
- Factory creation (`type_id::create()`) enables overrides
- Component hierarchy construction

1. Introduction to UVM (8/8)

- Phase execution demonstration
- Component creation and connection
- UVM report summary

2. UVM Class Hierarchy (1/3)

- Object vs Component
- `uvm_object` - Base for transactions, sequences
- `uvm_component` - Base for testbench components
- Lifetime differences
- Hierarchy differences
- Component Classes

2. UVM Class Hierarchy (2/3)

- `uvm_test` - Top-level test class
- `uvm_env` - Verification environment
- `uvm_agent` - Agent containing driver, monitor, sequencer
- `uvm_driver` - Drives transactions to DUT
- `uvm_monitor` - Observes DUT behavior
- `uvm_sequencer` - Manages sequence execution

2. UVM Class Hierarchy (3/3)

- Object Classes
- `uvm_sequence_item` - Base transaction class
- `uvm_sequence` - Sequence of transactions
- Class relationships and inheritance

3. UVM Phases (1/8)

- Build-Time Phases (Top-down)
- `build_phase()` - Component construction
- `connect_phase()` - Component connections
- `end_of_elaboration_phase()` - Final setup
- `start_of_simulation_phase()` - Simulation start
- Run-Time Phases (Bottom-up)

3. UVM Phases (2/8)

- `pre_reset_phase()`, `reset_phase()`, `post_reset_phase()`
- `pre_configure_phase()`, `configure_phase()`, `post_configure_phase()`
- `pre_main_phase()`, `main_phase()`, `post_main_phase()`
- `pre_shutdown_phase()`, `shutdown_phase()`, `post_shutdown_phase()`
- Cleanup Phases (Bottom-up)
- `extract_phase()` - Extract results

3. UVM Phases (3/8)

- `check_phase()` - Final checks
- `report_phase()` - Generate reports
- `final_phase()` - Final cleanup
- Build-Time Phases: Top-down execution (parent before child)
- Run-Time Phases: Bottom-up execution (child before parent)
- Cleanup Phases: Bottom-up execution (child before parent)

3. UVM Phases (4/8)

- Phase Execution Order: Demonstrates all UVM phases
- Purpose: Component construction and setup
- Execution: Top-down (parent before child)
- Key: Use `super.build_phase(phase)` to maintain hierarchy
- Purpose: Test execution phases
- Execution: Bottom-up (child before parent)

3. UVM Phases (5/8)

- Key: Use objections in `main_phase()` to control simulation
- Purpose: Result extraction and reporting
- Execution: Bottom-up (child before parent)
- Key: Use for final verification and reporting
- Purpose: Get component identification
- Usage: Logging, debugging, hierarchical paths

3. UVM Phases (6/8)

- Build-time phases execute top-down (parent before child)
- Run-time phases execute bottom-up (child before parent)
- Cleanup phases execute bottom-up (child before parent)
- Phases enable structured testbench execution
- Build-time phases are functions (no delays allowed)
- Run-time phases are tasks (can have delays)

3. UVM Phases (7/8)

- Cleanup phases are functions (no delays allowed)
- Must call `super.phase_name(phase)` to maintain hierarchy
- Use `get_name()` and `get_full_name()` for component identification
- All build-time phases executed in order
- All run-time phases executed in order
- All cleanup phases executed in order

3. UVM Phases (8/8)

- Phase execution logging

4. UVM Reporting (1/8)

- UVM Messaging System
- Severity levels: FATAL, ERROR, WARNING, INFO, DEBUG
- Verbosity levels: UVM_LOW, UVM_MEDIUM, UVM_HIGH, UVM_FULL, UVM_DEBUG
- Message formatting
- Hierarchical reporting
- Severity Levels: INFO, WARNING, ERROR, FATAL

4. UVM Reporting (2/8)

- Verbosity Levels: UVM_LOW, UVM_MEDIUM, UVM_HIGH, UVM_FULL, UVM_DEBUG
- Message Formatting: Formatted messages with data values
- Hierarchical Reporting: Component context in messages
- Purpose: Print formatted messages with severity
- Parameters:
- `TAG`: Message category/tag

4. UVM Reporting (3/8)

- `Message`: Text to display
- `Verbosity`: For `uvm_info` only (UVM_LOW, UVM_MEDIUM, etc.)
- Usage: Standard UVM messaging
- Purpose: Include data values in messages
- Uses: `$sformatf()` for string formatting
- Key: Combine UVM macros with SystemVerilog formatting

4. UVM Reporting (4/8)

- Purpose: Control message visibility
- Usage: Set in `build_phase()` or test
- Levels: UVM_LOW < UVM_MEDIUM < UVM_HIGH < UVM_FULL < UVM_DEBUG
- Purpose: Include component hierarchy in messages
- Uses: `get_full_name()` for hierarchical path
- INFO: Informational messages (non-critical)

4. UVM Reporting (5/8)

- WARNING: Potential issues (non-fatal)
- ERROR: Actual errors (may stop simulation)
- FATAL: Fatal errors (stops simulation immediately)
- UVM_LOW: Always displayed (default)
- UVM_MEDIUM: Medium detail (default for most messages)
- UVM_HIGH: High detail

4. UVM Reporting (6/8)

- UVM_FULL: Full detail
- UVM_DEBUG: Debug-level detail
- Severity indicates message importance
- Verbosity controls message visibility
- Messages include component hierarchy context
- Reporting enables debugging and monitoring

4. UVM Reporting (7/8)

- `uvm_info` is most common (with verbosity control)
- `uvm_warning` for non-critical issues
- `uvm_error` for errors (may stop simulation)
- `uvm_fatal` for fatal errors (stops simulation)
- Use `$sformatf()` for formatted messages with data
- Different severity levels demonstrated

4. UVM Reporting (8/8)

- Verbosity control demonstrated
- Formatted messages with data
- Hierarchical message context

5. ConfigDB (1/7)

- Configuration Database Basics
- Setting configuration
- Getting configuration
- Configuration hierarchy
- Configuration objects
- Setting Configuration: Global and component-specific

5. ConfigDB (2/7)

- Getting Configuration: Hierarchical lookup
- Configuration Objects: Complex configuration data
- Scalar Values: Simple configuration values
- Purpose: Store configuration in database
- Syntax: `uvm_config_db#(type)::set(contxt, path, field, value)`
- Parameters:

5. ConfigDB (3/7)

- `contxt`: Context (usually `this` in test)
- `path`: Component path (`"*"` for global,
  `"comp_name"` for specific)
- `field`: Configuration field name (string)
- `value`: Value to store
- Usage: Set in test's `build_phase()` before creating
  components
- Purpose: Retrieve configuration from database

5. ConfigDB (4/7)

- Syntax: `uvm_config_db#(type)::get(contxt, path, field, var)`
- Returns: `1` if found, `0` if not found
- Usage: Get in component's `build_phase()`
- Key: Always check return value, provide defaults if not found
- Purpose: String representation of configuration
- Usage: Debugging, logging

5. ConfigDB (5/7)

- ConfigDB enables flexible test configuration
- Configuration can be set globally or per-component
- Components look up configuration in `build_phase()`
- Hierarchical paths enable component-specific config
- Set before create: Configuration must be set before components are created
- Get in build_phase: Components get configuration in their `build_phase()`

5. ConfigDB (6/7)

- Check return value: Always check if `get()` succeeded
- Provide defaults: Use default values if configuration not found
- Type safety: ConfigDB is type-safe (uses template parameter)
- Configuration setting and getting
- Hierarchical configuration lookup
- Component-specific configuration

5. ConfigDB (7/7)

- Configuration object usage

6. Factory Pattern (1/6)

- Factory Pattern Basics
- Factory registration
- Factory creation
- Type override
- Instance override
- Factory Registration: Automatic registration via
  `uvm_object_utils`

6. Factory Pattern (2/6)

- Factory Creation: Using `type_id::create()`
- Type Override: Substituting base class with extended class
- Instance Override: Overriding specific instances
- Purpose: Automatically register class with factory
- Usage: Must be inside class definition
- Key: Enables factory-based creation and overrides

6. Factory Pattern (3/6)

- Purpose: Create objects/components via factory
- Syntax: `ClassName::type_id::create(name, parent)`
- Key: Factory applies overrides automatically
- Purpose: Replace all instances of base class with extended class
- Usage: Set in test's `build_phase()` before creating components
- Effect: All `BaseDriver::type_id::create()` calls create `ExtendedDriver`

6. Factory Pattern (4/6)

- Purpose: Replace specific instance with extended class
- Usage: Set in test's `build_phase()` before creating components
- Effect: Only specified instance is overridden
- Purpose: Get type wrapper for override operations
- Usage: Used in override methods
- Factory enables centralized object creation

6. Factory Pattern (5/6)

- Type override affects all instances
- Instance override affects specific instances
- Overrides enable test customization
- Factory macros (`uvm_object_utils`, `uvm_component_utils`) register classes
- Factory creation (`type_id::create()`) applies overrides automatically
- Set overrides before create: Overrides must be set before creating components

6. Factory Pattern (6/6)

- Type override: Affects all instances of the base class
- Instance override: Affects only the specified instance
- Factory creation demonstrated
- Type override demonstrated
- Instance override demonstrated
- Override effects verified

7. Objections (1/6)

- Objection Mechanism
- Raising objections
- Dropping objections
- Phase completion
- Simulation control
- Raising Objections: Keep simulation running

7. Objections (2/6)

- Dropping Objections: Allow phase completion
- Multiple Objections: Per-component objection tracking
- Coordinated Completion: Phase completes when all objections dropped
- Purpose: Keep phase running until objection is dropped
- Usage: Call at start of `run_phase()` or other run-time phases
- Parameters:

7. Objections (3/6)

- `this`: Component raising objection
- `description`: Optional description for named objections
- Key: Must raise objection if doing work in run-time phase
- Purpose: Signal that work is complete
- Usage: Call when work in phase is finished
- Key: Must drop objection when work completes

7. Objections (4/6)

- Pattern: Raise at start, drop when complete
- Key: Ensures phase doesn't end prematurely
- Purpose: Track multiple independent tasks
- Usage: Use named objections for different tasks
- Key: Each named objection is tracked separately
- Raise objection to keep phase running

7. Objections (5/6)

- Drop objection when work completes
- Phase completes when all objections dropped
- Multiple objections per component supported
- Raise at start: Call `raise_objection()` at beginning of work
- Drop when done: Call `drop_objection()` when work completes
- Phase completion: Phase ends when all objections are dropped

7. Objections (6/6)

- Named objections: Use for tracking multiple tasks
- Must match: Each `raise_objection()` must have matching `drop_objection()`
- Objection raising and dropping
- Phase completion control
- Multiple objection handling
- Coordinated phase completion

8. UVM Functions and Methods Reference

- Purpose: Register class with UVM factory
- Location: Inside class definition
- Enables: Factory creation and overrides
- Syntax: `ClassName::type_id::create(name, parent)`
- Returns: Handle to created object/component
- Key: Factory applies overrides automatically

Hands-on examples

Module 3 self-check

```
$ ./scripts/module3.sh --check
```

Exercise scaffold

```
./scripts/module3.sh --scaffold
```

Practice & assessment

Exercises

- Class Hierarchy
- Phases
- Reporting
- ConfigDB
- Factory

Exercises

- Objections

Assessment checklist

- Understands UVM class hierarchy
- Masters UVM phases
- Uses UVM reporting effectively
- Configures components using ConfigDB
- Understands factory pattern
- Controls simulation with objections

Summary & next steps

- Master UVM class hierarchy and phases
- Complete module3/CHECKLIST.md
- Run ./scripts/module3.sh --check
- Next: Next module in course